S

SkyBook

Decentralised Flight Ticketing Protocol on BOT Chain

WHITEPAPER | VERSION 1.0
October 2026 · BOT Chain · Chain ID 968

PROTOCOL OVERVIEW

SkyBook is a decentralised flight ticketing protocol deployed on BOT Chain. Admin-approved airlines list flights with routes, departure times, seat counts, and ticket prices. Passengers purchase tickets by sending the exact ETH price; revenue accumulates in the contract against each airline's address. Airlines claim their earnings at any time. Passengers may cancel tickets up to 24 hours before departure for a full refund. Every ticket is recorded on-chain, creating an immutable, tamper-proof booking ledger.

A technical overview of the protocol, its design, core functions, security properties, and roadmap.

1. Problem Statement

- Opaque pricing — airline and OTA pricing algorithms prevent fare transparency.
- Overbooking — centralised systems allow airlines to oversell beyond seat capacity.
- Refund friction — passengers often wait weeks for refunds routed through payment processors.
- Ticket scalping — resale markets inflate prices without airline consent or revenue share.
- Single-point booking failure — centralised GDS downtime blocks all ticket sales.

2. Protocol Design

2.1 Roles

`Role`
Permissions

Admin

Register / deregister airlines

Airline

Add flights (route, departure, price, seats); claim earned revenue

`Role`
Permissions

Passenger

Book a ticket; cancel a ticket (>24 h before departure)

2.2 Seat Availability

The contract enforces f.sold < f.seats on every book() call. Overbooking is impossible — seat availability is determined by immutable on-chain state.

2.3 Revenue Accounting

earned[airline] accumulates ETH from ticket sales. On cancellation, the amount is deducted back. Airlines call claim() to withdraw their full balance in one transaction.

3. Contract Architecture

3.1 Data Model

Flight: airline address, route (string), departure (unix timestamp), price (wei), seats (total), sold (count).

Ticket: flightId, owner address, cancelled flag.

earned mapping: airline address => accrued ETH (wei).

3.2 Write Functions

`Function`
`Role`
Description

`addAirline(addr)`
Admin

Grants airline role

`addFlight(route, departure, price, seats)`

Airline

Creates a new flight listing

`book(flightId)`
Passenger

Purchases ticket — exact ETH required

`cancel(ticketId)`
Ticket Owner

Full refund if >24 h before departure

claim()

Airline

Withdraws all accrued ticket revenue

4. Security Properties

4.1 Exact Payment

require(msg.value == f.price) rejects any overpayment or underpayment — no change, no rounding errors.

4.2 Cancellation Window

require(block.timestamp + 1 days < f.departure) enforces a hard 24-hour cancellation cutoff. The contract cannot be manipulated to grant last-minute refunds.

4.3 Revenue Isolation

earned[airline] tracks each airline's balance independently. A malicious airline calling claim() cannot drain another airline's funds.

4.4 Departure Cutoff

require(block.timestamp < f.departure) in book() prevents ticket purchases for already-departed flights.

5. Events & Data Access

The public flights and tickets arrays allow any frontend to enumerate all flights and tickets without custom view functions. The public earned mapping exposes airline revenue balances.

6. Deployment

`Parameter`
Value

`Chain`
BOT Chain (Chain ID 968)

`RPC`
https://rpc.botchain.ai

`Explorer`
https://scan.botchain.ai

Cancellation window

More than 24 hours before departure

7. Roadmap

Milestone	Description
v1.1	ERC-721 NFT tickets for transferable boarding passes
v1.2	Partial refund on late cancellation (e.g. 50% within 24 h)
v1.3	Seat-class differentiation (economy / business)
v2.0	Multi-hop itinerary booking across airlines